DeepL
Subscribe to DeepL Pro to translate larger documents.
Visit www.DeepL.com/pro for more information.

Contest Guidelines V1

I. Problem Title

A Time-Driven Design

Partitioner II. Problem

Background

As the demand for larger logic capacity and higher performance in FPGAs continues to grow, traditional monolithic silicon technology has reached bottlenecks in yield and process technology.To overcome these limitations, the industry has begun exploring a "divide and conquer" approach, which involves partitioning large-scale logic into several smaller units and interconnecting them via 2.5D packaging technology. This stacked silicon interconnect technology not only increases the number of logic units but also significantly reduces power consumption and thermal issues, making it a key method for breaking through the limitations of Moore's Law.

As shown in Figure 1, in 2.5D packaging, the individual dies are not stacked vertically but are arranged side-by-side on a common silicon interposer. This interposer is typically made of silicon and has a coefficient of thermal expansion similar to that of the dies, which helps reduce thermal stress and improve the mechanical stability of the package.Figure 2 illustrates the interconnect process between dies, where the connections between each die and the silicon interposer utilize micro-bump technology. The I/O signals from each die are bonded to the interposer via micro-bumps, and then routed to the corresponding ports on other dies via interconnects within the interposer.

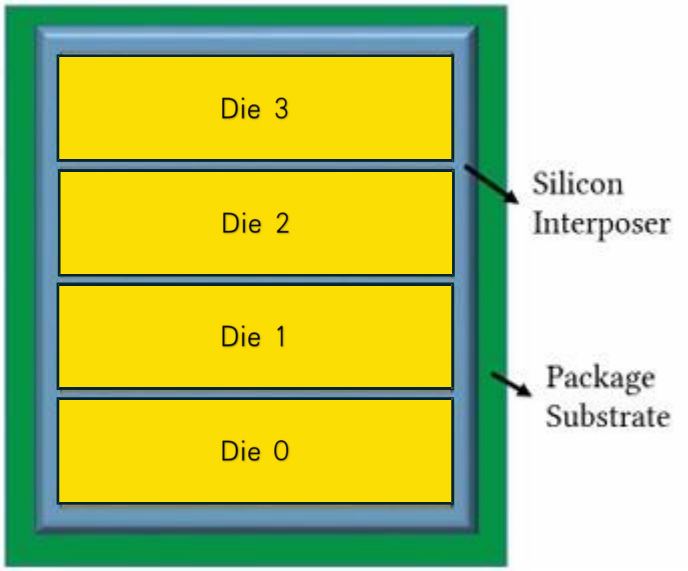

Figure 1. Schematic diagram of a 2.5D FPGA package

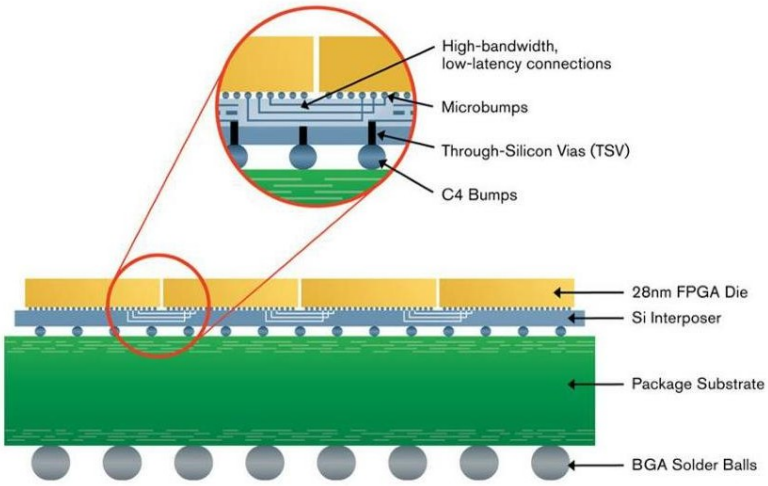

Figure 2. Schematic of Inter-Die Interconnection

To accommodate the characteristics of 2.5D FPGAs, a core component of FPGA implementation tools is design partitioning. This process allocates submodules within the design across individual dies based on physical constraints (including the number of on-die resources, the number of inter-die interconnects, partitioning constraints, I/O constraints, etc.) and timing constraints. Compared to traditional partitioning methods, a timing-driven partitioning scheme requires achieving a timing-optimized partitioning result while satisfying all physical constraints.

This competition is primarily aimed at students majoring in integrated circuits, computer science, mathematics, and related fields. Participants are expected to possess relevant knowledge of integrated circuits and EDA tools, as well as engineering software development capabilities. Students with experience in timing-driven partitioning or 2.5D FPGA system design and implementation are particularly well-suited for this challenge.

III. Problem Description

This competition requires teams to propose a partitioning scheme based on the given gate-level netlist, 2.5D FPGA architecture information, physical constraints, and timing constraints. Timing performance and program execution time will serve as the evaluation criteria for partitioning quality.

Constraints:

1. Each test case will specify timing constraints. For example, the design must meet a target frequency (e.g., 500 MHz).
2. The partitioning result must not exceed the maximum utilization of logic resources (LUTs, FPs, DSPs, etc.) contained in each die.
3. Interconnects between dies may only carry a single cut net in any direction; the partitioning result must not exceed the maximum utilization of inter-die interconnects.
4. The partitioning algorithm must not alter the input files or data, modify the netlist structure, or change the positions of fixed physical cells

Input Files:

Each test case includes the following input files:

1. Gate-level netlist (.edf)

   This file contains the gate-level netlist, which is organized hierarchically by module.

2. Timing Constraint File (.sdc)

   This file contains all relevant timing constraints.

3. Physical constraint file (.txt)

   This file maps specified cells and ports to a specific die, in the following format:

```
cell_a    die0
cell_b    die1
cell_c    die2
port_a    die0
port_b die1
```

Each line in the file represents a constraint. Each line contains the names of the cell and port, along with the die to which they are constrained. All submodules contained within that cell must be constrained to the specified die.

4. 2.5D FPGA Physical Description File (.xml)

This file describes the number of logic resources and maximum utilization within each die of the target 2.5D FPGA, as well as the number of interconnects between dies and their maximum utilization. The format is as follows; resource types are not limited to LUTs, FFs, etc.

```
<?xml version="1.0" encoding="UTF-8"?>
<fpga>
  <!-- Define the logic resources for each die -->
  <dies>
    <!-- Die name -->
    <die name="die0">
      <!-- Resource type, quantity, and maximum utilization -->
      <resource type=''lut''                    count="150000''
      util=''0.8''  />
      <resource type="ff''   count="150000''    util=''0.8''  />
      <resource type="dsp"   count=  “150000”    util="0.8" />
      <resource type="bram"  count="150000"     util="0.8" />
      <resource type="carry"  count="150000"    util="0.8" />
    </die>
    <die name="die1">
      <resource type=''lut''                    count="150000''
      util=''0.8''  />
      <resource type="ff''   count="150000''    util=''0.8''  />
      <resource type="dsp"   count=  “150000”    util="0.8" />
      <resource type="bram"  count="150000"     util="0.8" />
```

```
        </die>
        <die name="die2">
            <resource type=''lut''                          count="150000''
           util=''0.8''  />
            <resource type="ff''    count="150000''        util=''0.8''  />
            <resource type="dsp"   count=     “150000”        util="0.8" />
           <resource type="bram"   count="150000"      util="0.8" />
        </die>
    </dies>
    <interconnections>
        <!-- Number of interconnections between die0 and die1 and maximum
       utilization-->
        <interconnection dies="die0 die1"  count="10000"     util="0.6" />
        <interconnection dies="die1 die2"  count="10000"     util="0.6" />
    </interconnections>
</fpga>
```

5. Timing file (.lib)

This file describes the timing information for various cell types. The following example shows the timing information for LUT6.

```
cell(LUT6) { area :
   1;
    pin(I0,I1,I2,I3,I4,I5) {
        capacitance : INPUT_CAP; direction :
       input;
    }
    lut(L) {
       input_pins: "I0 I1 I2 I3 I4 I5";
    }
    pin(O) {
        direction: output; function:
       "L";
        timing() {
                timing_type : combinational;
                        intrinsic_rise : 0.043;
                         intrinsic_fall : 0.043;
               related_pin : "I0";
        }
        timing() {
                timing_type: combinational;
                        intrinsic_rise: 0.043;
                         intrinsic_fall : 0.043;
               related_pin : "I1";
        }
        timing() {
```

```
                timing_type : combinational;
                        intrinsic_rise : 0.043;
                        intrinsic_fall : 0.043;
                related_pin : "I2";
        }
        timing() {
                timing_type: combinational;
                        intrinsic_rise: 0.043;
                        intrinsic_fall : 0.043;
                related_pin : "I3";
        }
        timing() {
                timing_type: combinational;
                        intrinsic_rise: 0.043;
                        intrinsic_fall : 0.043;
                related_pin : "I4";
        }
        timing() {
                timing_type: combinational;
                        intrinsic_rise: 0.043;
                        intrinsic_fall : 0.043;
                related_pin : "I5";
        }
    }
}
```

Explanation of Interconnect Delay Calculation:

The initial delay value for all interconnects is uniformly estimated at 0.2 ns. For each additional input port on an interconnect, the delay increases by 0.1 ns. Each time an interconnect crosses a die, the delay increases by 2 ns. Taking Figure 3 as an example, the final delay value for interconnect a should be $0.2 + 3 \times 0.1 + 2 \times 2$.

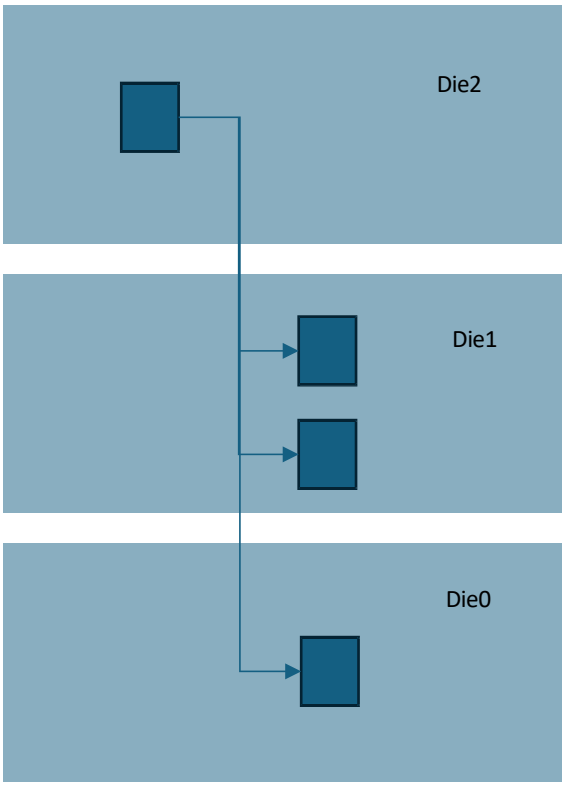

Figure 3: Net delay calculation

Output:

Output partitioning file (.txt) in the following format:

```
cell_a  die0
cell_b  die1
cell_c  die2
port_a  die0
port_b die1
```

Each line of the file represents a partitioning result. Each line contains the cell

name and the die name to which it belongs. IV. Scoring Criteria

This competition will provide several test cases of varying difficulty. The difficulty of the "Hidden Cases" is roughly equivalent to that of the "Public Cases." Rankings will be determined based on the total score across all cases during evaluation. The competition will provide an evaluation program for participating teams to calculate TNS and WNS.

1. Scoring Criteria:

1) If the output violates any constraint, the test case *is scored as a + β + Y* points.

2) Each test case is scored according to the following function, and rankings are determined based on the total score across all test cases.

$$Score = a + \left|\frac{WNS}{WNS_{MAX}}\right| + \beta + \left|\frac{TNS}{TNS_{MAX}}\right| + Y + \frac{Runtime}{Runtime_{MAX}}$$

aβand Y are weights; specific values will be provided in the following explanation.

WNS (Worst-Case Negative Timing Relaxation): Reflects the severity of timing violations on the critical path in the design; a larger absolute value indicates a more severe timing issue.

WNS_max: The worst WNS among all contest results (e.g., -200 ps)

TNS (Total Negative Timing Slack): Reflects the cumulative severity of timing violations across all timing paths; a larger absolute value indicates poorer overall timing convergence.

TNS_max: The worst TNS among all contest results (e.g., -5000 ps)

Runtime: The time taken for the optimization process,

measured in seconds.

Runtime_max: The longest runtime among all competition results (e.g., 24 hours = 86,400 seconds)

2. Submission Guidelines:

An executable file must be provided, along with a presentation (PPT) explaining the solution. Source code is encouraged. Any third-party libraries used must also be provided.

V. References

[1] Chirag Ravishankar; Dinesh Gaitonde; Trevor Bauer, “Placement Strategies for 2.5D FPGA Fabric Architectures”, 2018 28th International Conference on Field Programmable Logic and Applications (FPL)

[2] G. Karypis, R. Aggarwal, V. Kumar, and S. Shekhar, ‘Multilevel Hypergraph Partitioning: Application in VLSI domain’, Proc. ACM/IEEE DAC, June 1997.

[3] OpenSTA: https://github.com/parallaxsw/OpenSTA

*For any questions not addressed in this contest guide, please attend the contest briefing session or refer to the contest Q&A document.